Oil, Vinegar, and Sparks: Key Recovery from UOV via Single Electromagnetic Fault Injection

Fabio Campos[1], Daniel Hahn[2], Daniel Könnecke[2] and Marc Stöttinger[2]

[1] Darmstadt University of Applied Sciences, Germany
campos@sopmac.de
[2] RheinMain University of Applied Sciences Wiesbaden, Germany
daniel.hahn@hs-rm.de daniel@koe-mail.com marc.stoettinger@hs-rm.de

Abstract. In this work, we present a practical fault-injection attack against the current Unbalanced Oil and Vinegar signature scheme non-deterministic implementation that enables complete secret key recovery from a single faulty signature, given one prior fault-free signature. By applying fault injection to disrupt the mixing of the vinegar and oil components during signature generation, the secret key can be recovered directly from the faulty signature. We validate the attack experimentally on a real device using an electromagnetic fault injection setup, establishing the required glitch in practice rather than through simulation. Our experimental results indicate that a single execution of the attack achieves a success rate exceeding 90%. Furthermore, we demonstrate the real-world feasibility of the attack by transferring the attack parameters to an architecturally equivalent target, requiring only minor recalibration. To the best of our knowledge, this is the first work to demonstrate an electromagnetic fault-injection attack in the context of multivariate-based signature schemes. Additionally, we discuss possible countermeasures to protect implementations of UOV-based schemes against such physical attacks.

Keywords: UOV · Fault Injection Attack · EMFI

1 Introduction

Post-quantum cryptography has reached practical maturity with the publication of standards FIPS 203, FIPS 204, and FIPS 205. However, standardization efforts continue for alternative schemes offering different tradeoffs in key sizes, signature lengths, and computational efficiency. Multivariate signature schemes are particularly relevant due to their compact signatures and fast verification, properties explicitly sought in NIST's 2022 call for additional signature schemes[1].

The path toward multivariate signature standardization has been turbulent. Rainbow reached the third round of NIST's initial process but was eliminated following a powerful attack [Beu22]. This attack renewed interest in the unbalanced oil and vinegar (UOV) scheme [Pat97]. While Rainbow's optimizations improved efficiency, they also introduced vulnerabilities that UOV avoids. UOV's simpler structure has withstood attacks for over two decades.

*Author list in alphabetical order; see https://www.ams.org/profession/leaders/CultureStatement04.pdf. This work has been supported by the German Federal Ministry of Research, Technology and Space (BMFTR) under the projects Parfait (16KIS2144), QUDIS (16KIS2089), SASPIT (16KIS1858), and DI-SIGN-HEP (16KIS2072).
Date of this document: 2026-01-30.

[1] https://csrc.nist.gov/projects/pqc-dig-sig

In response to NIST's call for additional post-quantum digital signature schemes, forty submissions were received, ten of which employed multivariate techniques. Three schemes (3WISE, DME, and HPPC) were quickly broken, while the remaining seven candidates (MAYO [BCC+23], PROV [GCF+23], QR-UOV [FIH+23], SNOVA [WCD+23], TUOV [DGG+23], UOV [BCD+23], VOX [PCF+23]) all rely on principles derived from the Unbalanced Oil and Vinegar (UOV) construction. In October 2024, NIST advanced four of these schemes to the second round of evaluation: MAYO, QR-UOV, SNOVA, and UOV. The security of the remaining multivariate candidates encompasses both mathematical resilience against classical and quantum cryptanalysis and the physical robustness of their implementations. Both requirements are critical as these schemes are deployed on embedded and constrained platforms, necessitating a rigorous evaluation under realistic implementation threats. Consequently, the research community has placed a significant focus on the physical security of UOV-based signatures, ensuring they can withstand side-channel and fault-based attacks in practical environments.

In this work, we present a practical single-fault key recovery attack on the current UOV implementation using electromagnetic fault injection, highlighting the need for effective countermeasures.

Related Work The literature on the physical security of multivariate cryptography has expanded significantly since the inaugural analysis of implementation-level vulnerabilities in multivariate signature schemes [HTS11]. Subsequent research has explored a wide spectrum of both passive side-channel analysis and active physical attacks. Specifically, the first single-trace side-channel attack against UOV was introduced in [ACK+23], which utilized a single side-channel trace to recover all vinegar variables used during the signing process. Recent work [BSK25] demonstrated exploits on deterministic UOV variants, though these require a differential approach using a correct and faulty signature pair. Other methodologies have targeted the underlying primitives, such as bypassing a hash call to fix vinegar values [AMSU24], or utilizing algebraic analysis of perturbed executions on both UOV and Rainbow [SK20]. Furthermore, the security of LUOV and MAYO has been scrutinized through hybrid attacks [MIS20] and protected hardware evaluations [SMA+24, JD24].

Despite this breadth of research, many existing exploits remain largely theoretical [HTS11, KL19, SK20, AMSU24] or have been validated only through simulation [AKKM22, FKNT22]. To consolidate these findings, [ACK25] recently provided a comprehensive survey of physical threats and countermeasures for UOV-based NIST candidates. Collectively, these studies demonstrate that the security of multivariate schemes in practice crucially depends on the resilience of their implementations against physical tampering.

Contribution In this work, we present a practical fault-injection attack against the UOV signature scheme. Focusing on the current UOV implementation targeting NIST Security Level 1 (`uov-Ip`), we show that complete secret key recovery can be achieved from a single fault injection and that the underlying technique generalizes to other parameter sets. To the best of our knowledge, this work presents the first practical realization of a skip-addition attack [ACK25] on UOV and the first demonstration of such an attack using real electromagnetic fault injection.

Our attack proceeds in two phases. First, a single fault injected during signature generation enables the recovery of one oil-space vector. Second, we adapt the Kipnis–Shamir–based approach of [ACK+23] to recover a second oil-space vector. Unlike [ACK+23], which evaluates the attack on reduced parameter sets, our attack applies directly to the `uov-Ip` parameter set. For UOV, these two vectors suffice to reconstruct the entire oil space, which is equivalent to the full secret key, using an efficient reconciliation technique based solely on solving linear systems.

We implement and evaluate the attack on an STM32F415 ARM Cortex-M4 microcontroller. For electromagnetic fault injection, we develop a precise experimental setup based on a ChipSHOUTER[2] and a custom Raspberry Pi Pico–based pulse generator. This configuration enables nanosecond-resolution control over fault timing and duration, ensuring the attacks are highly repeatable. Critically, we examine the feasibility of the attack in a real-world scenario. To validate the feasibility in real world scenario, we transfer the attack to an architecturally equivalent target with minimal effort.

Finally, we discuss potential countermeasures. The complete codebase required to reproduce the attack, including fault injection, secret key recovery, and experimental artifacts, is available:

`https://github.com/UOV-EMFI-Fault-Attack/OVER`.

Organization The remainder of this paper is organized as follows. Section 2 establishes the mathematical foundations of UOV and the algebraic properties enabling key recovery from leaked oil or vinegar vectors. In Section 3, we detail our attack methodology and its application to the current `uov-Ip` implementation provided by the pqm4 project [KRSS19]. Section 4 describes the experimental setup and electromagnetic fault injection (EMFI) platform used, while Section 5 provides a detailed breakdown of the attack phases. Section 6 presents the results and evaluates real-world attack feasibility through statistical success rates. Potential countermeasures and the impact of security engineering practices are discussed in Section 7, followed by concluding remarks and future work in Section 8.

2 Recap on Unbalanced Oil and Vinegar Signature Scheme

In this section, we recall only the core properties of UOV that are required to understand our attack and its implications. We explicitly connect the underlying mathematical construction to the corresponding steps in the pseudocode. For a comprehensive treatment of UOV and its variants, including parameter choices and security considerations, we refer the reader to [BCD+23].

In multivariate cryptography, multivariate quadratic maps $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$ are the primary objects. Generally speaking, finding a solution $\mathbf{s} \in \mathbb{F}_q^n$ to a given target $\mathbf{t} \in \mathbb{F}_q^m$ such that $\mathcal{P}(\mathbf{s}) = \mathbf{t}$ is a hard problem, also known as the MQ Problem. If $n \sim m$, it is believed to be exponentially hard, even for large-scale quantum computers. However, it can be solved in polynomial time in the very under- or overdetermined case, where $m \geq n(n+1)/2$ or $n \geq m(m+1)$. By incorporating a secret trapdoor into the public map $\mathcal{P}$, it is possible to efficiently solve this task and develop a signature scheme for it. In UOV, this trapdoor is a m-dimensional linear subspace $O \subset \mathbb{F}_q^n$, which is the oil space. It has the property $\mathcal{P}(\mathbf{o}) = 0$ for all $\mathbf{o} \in O$. To sign a message μ, the signer first computes a hash of the message and a random `salt` to a target value $\mathbf{t} = h(\mu||\texttt{salt})$. Finding a preimage $\mathbf{s} \in \mathbb{F}_q^n$ using $P(\mathbf{s}) = \mathbf{t}$ is the main step in computing a signature. To find such a vector s, the signer samples a random vector $\mathbf{v}$, and then solves the linear system of equations

$$\mathcal{P}(\mathbf{v} + \mathbf{o}) = \mathcal{P}(\mathbf{v}) + \mathcal{P}(\mathbf{o}) + \mathcal{P}'(\mathbf{v}, \mathbf{o}) = \mathbf{t} \tag{1}$$

for $\mathbf{o} \in O$. The differential of $\mathcal{P}$ is a bilinear and symmetric map $\mathcal{P}' : \mathbb{F}_q^n \times \mathbb{F}_q^n \to \mathbb{F}_q^m$, as illustrated by the aforementioned equation [Beu21]. Considering the oil vector $\mathbf{o}$ as a linear combination of its m basis vectors given in O guarantees that $\mathcal{P}(\mathbf{o}) = \mathbf{0}_m$ and makes it evident that

$$\mathcal{P}'(\mathbf{v}, \mathbf{o}) = \mathbf{t} - \mathcal{P}(\mathbf{v}) \tag{2}$$

[2] `https://github.com/newaetech/ChipSHOUTER`

Algorithm 1 UOV Signature Generation

Input: Expanded secret key $esk = (\texttt{seed}_{\text{sk}}, \mathbf{O}, \{\mathbf{P}_i^{(1)}, \mathbf{S}_i\}_{i=1}^m)$, message μ
Output: Signature σ or failure

1: $\texttt{salt} \xleftarrow{\$} \{0,1\}^{\text{salt\_len}}$
2: $\mathbf{t} \leftarrow \texttt{H}(\mu \parallel \texttt{salt})$ ▷ $\mathbf{t} \in \mathbb{F}_q^m$
3: **for** $ctr = 0$ **to** 255 **do**
4: $\quad \mathbf{v} \leftarrow \texttt{Expand}_v(\mu \parallel \texttt{salt} \parallel \texttt{seed}_{\text{sk}} \parallel ctr)$ ▷ $\mathbf{v} \in \mathbb{F}_q^{n-m}$
5: $\quad \mathbf{L} \leftarrow \mathbf{0}_{m \times m}$
6: $\quad$ **for** $i = 1$ **to** m **do**
7: $\quad\quad$ Set row i of $\mathbf{L}$ to $\mathbf{v}^\top \mathbf{S}_i$
8: $\quad$ **end for**
9: $\quad$ **if** $\mathbf{L}$ is invertible **then**
10: $\quad\quad \mathbf{y} \leftarrow [\mathbf{v}^\top \mathbf{P}_i^{(1)} \mathbf{v}]_{i=1}^m$ ▷ $\mathbf{y} \in \mathbb{F}_q^m$
11: $\quad\quad$ Solve $\mathbf{L}\mathbf{x} = \mathbf{t} - \mathbf{y}$ for $\mathbf{x}$
12: $\quad\quad \mathbf{s} \leftarrow [\mathbf{v}, \mathbf{0}_m] + \bar{\mathbf{O}}\mathbf{x}$ ▷ $\mathbf{s} \in \mathbb{F}_q^n$
13: $\quad\quad \sigma \leftarrow (\mathbf{s}, \texttt{salt})$
14: $\quad\quad$ **return** σ
15: $\quad$ **end if**
16: **end for**
17: **return** $\perp$

is a system of m linear equations in m variables. The system parameters are selected such that a solution exists with high probability. If there is a solution, it can be quickly found; if not, the signer samples a new vector $\mathbf{v}$ and repeats the procedure. The core of the signature is created by the preimage $\mathbf{s} = \mathbf{v} + \mathbf{o}$ that is generated by the vinegar and oil vectors when combined.

Signing The signing algorithm is shown in Algorithm 1. After the target value $\mathbf{t}$ is derived and the salt $\texttt{salt}$ is generated, the vinegar vector $\mathbf{v}$ is sampled. It is important to note that the final m entries of $\mathbf{v}$ are set to zero, which facilitates a more efficient implementation. After getting $\mathbf{v}$, the linear part of the system given in Line 10 is found in Line 7 by computing the rest of $\mathcal{P}'(\mathbf{v}, \mathbf{o})$. Only the submatrix $\mathbf{P}_i^{(1)}$ is required to evaluate $\mathbf{y} = \mathcal{P}(\mathbf{v})$ in Line 9 because the final entries of $\mathbf{v}$ were selected to be zero. By solving the derived linear system, the coefficients $\mathbf{x}$ of the oil vector $\mathbf{o} = \bar{\mathbf{O}}\mathbf{x} = [\mathbf{O}\mathbf{x}, \mathbf{x}]$ are revealed. In conclusion, the signature $\mathbf{s} = [\mathbf{v}, \mathbf{0}_m] + \bar{\mathbf{O}}\mathbf{x}$ is obtained by adding the vinegar and oil vectors.

Remark on Minimal Leakage for Full Key Recovery As noted in [ACK25], the recovery of a single oil or vinegar vector is sufficient to reconstruct the complete secret key of most UOV-based schemes. This sensitivity ensures that any physical leakage occurring during the signing process poses a definitive threat to the system's long-term security.

3 Attack Design and Methodology

This section contains the theoretical basis of the attack and its practical application. The attack builds on the skip-addition concept introduced in [ACK25] and exploits specific properties of the reference implementation to enable recovery of the secret oil variables.

3.1 Attack Foundation

The goal of the attack is to manipulate the addition of oil and vinegar during signature generation, causing the secret oil vector to leak into the public signature output. The focus here lies on the composition of the signature $\sigma = (\mathbf{s}, \mathbf{salt})$, where $\mathbf{s}$ is computed as $\mathbf{v} + \mathbf{o}$. The attack prevents the use of $\mathbf{v}$ when mixing $\mathbf{v}$ with $\mathbf{o}$, after these values have been initialized. As a result, an incorrect signature $\mathbf{s}_f = \mathbf{o}_f$ is generated instead of the genuine signature $\mathbf{s}_g = \mathbf{v}_g + \mathbf{o}_g$. Since the fault is injected after initialization and only affects the addition step, the attack applies to both deterministic and non-deterministic signing procedures.

3.2 Practical Exploitation

Practical exploitation requires not only disrupting the addition $\mathbf{v} + \mathbf{o}$, but also extracting the oil vector from the resulting faulty signature. To achieve this, we target the critical signature composition step in the implementation. Specifically, our attack focuses on the `uov-Ip` reference implementation[3] from the pqm4 framework. Listing 1 presents the relevant code lines of the `ov_sign` function, where our skip-addition fault is injected during signature generation. Notably, the targeted code structure is consistent across all available UOV implementations, ensuring our findings generalize beyond this specific reference.

The computation of $\mathbf{s}_g = \mathbf{v}_g + \mathbf{o}_g$ proceeds in two steps: first, $\mathbf{v}$ is written to memory location $\mathbf{w}$, then $\mathbf{o}$ is added to this stored value. Specifically, a `memcpy` call copies the `vinegar` variable to the signature's final storage location $\mathbf{w}$ (see Line 11 in Listing 1). The number of bytes copied is defined by `_V_BYTE`, which varies with the UOV security level. During the faulted execution, skipping the `memcpy` prevents $\mathbf{v}_f$ from being written to $\mathbf{w}$.

$$\mathbf{s}_f = \mathbf{s}_g + \mathbf{o}_f. \tag{3}$$

This implies in practice that the faulty signature does not directly reveal the oil vector. Instead, it contains the bitwise XOR of the oil vector $\mathbf{o}_f$ and the pre-existing memory state of the signature $\mathbf{s}_g$. This memory state often proves predictable in practice. For instance, when multiple signatures are generated consecutively using the same signature pointer, the prior memory state is known as the previously generated signature. Since that prior signature is public, the attacker can easily compute the bitwise XOR of the previous signature and the faulty output to isolate the oil vector:

$$\mathbf{s}_g - \ \mathbf{s}_f = \mathbf{s}_g - \mathbf{s}_g + \mathbf{o}_f = \mathbf{o}_f. \tag{4}$$

The extracted oil vector $\mathbf{o}_f$ from the faulty signature generation σ_f enables complete private key recovery. Following [ACK+23], we apply a combination of the Kipnis-Shamir attack and the reconciliation attack to reveal the complete secret key.

While our experiments use the prior fault-free signature to predict pre-existing memory state

Remark on Memory Predictability While in our case a prior fault-free signature is used to predict pre-existing memory state, in general this memory predictability can also arise from various other sources, such as static value initialization or deterministic memory management routines. In these cases, a single faulty signature on an arbitrary message μ' suffices for key recovery.

During our investigation, we observed that even certain security engineering practices, in particular the zeroization of sensitive stack data as discussed in [OBG+23], might make the suggested skip addition attack easier to carry out if applied incorrectly. Specifically, zeroizing the memory locations holding the vinegar vector or the intermediate addition

[3] Commit: *a24bb4b*

result simplifies the attack, as a fault skipping the assignment of the vinegar variables directly exposes the oil vector.

Remark on Deterministic Settings In deterministic signature generation with fixed vinegar values [BSK25], the difference $\sigma_g - \sigma_f$ is invariant to the message relationship $\mu = \mu'$. Since deterministic generation ensures $\mathbf{s}_f$ and $\mathbf{s}_g$ share identical vinegar and oil components, we obtain[4]:

$$\mathbf{s}_f = \mathbf{s}_g + \mathbf{o}_f = \mathbf{v} + \mathbf{o} + \mathbf{o} = \mathbf{v}. \tag{5}$$

Consequently, the subtraction yields the same result as in the non-deterministic case:

$$\mathbf{s}_g - \mathbf{s}_f = \mathbf{v} + \mathbf{o} - \mathbf{v} = \mathbf{o}. \tag{6}$$

4 Hardware and Physical Setup

This section outlines the hardware platform and physical setup used for the experimental evaluation of the proposed attack, including the target device and the electromagnetic fault-injection infrastructure.

4.1 Target Device Specifications

Investigations are conducted on an STM32F415 ARM Cortex-M4 target mounted on a CW308 UFO board. This target is widely used in physical security research and provides a reproducible environment compatible with the pqm4 framework, making it well suited for evaluating practical attacks.

Due to the limited on-chip SRAM of the STM32F415 (192 KB), executing the UOV key generation on the target is infeasible, as the private key exceeds the available memory. Instead, the key pair is generated in advance on an x86 system using the `pqov` implementation [Pqo25] and embedded into the firmware as a constant array via a header file. These

[4] Under UOV's binary field arithmetic, $\mathbf{o} + \mathbf{o} = 0$, causing $\mathbf{s}_f$ to contain only $\mathbf{v}$.

Listing 1: Reduced excerpt of the pqm4 signature generation code showing the fault-injection target. The targeted `memcpy` operation is highlighted.

```
int ov_sign( uint8_t *signature, const sk_t *sk, const uint8_t *message, size_t mlen )
    {
    // ...
    uint8_t vinegar[_V_BYTE];
    // ...

    hash_final_digest( vinegar, _V_BYTE, &h_vinegar);   // H(M||salt||sk_seed||ctr)
    // ...

    //  w = T^-1 * x
    uint8_t *w = signature;     // [_PUB_N_BYTE];
    // identity part of T.
    memcpy( w, vinegar, _V_BYTE );
    memcpy( w + _V_BYTE, x_o1, _O_BYTE );
    // Computing the O part of T.
    gfmat_prod(y, sk->O, _V_BYTE, _O, x_o1 );
    gf256v_add(w, y, _V_BYTE );
    // return: signature <- w || salt.
    memcpy( signature + _PUB_N_BYTE, salt, _SALT_BYTE );

    return 0;
}
```

constants are placed by the compiler in the read-only data section and thus reside in flash memory, avoiding additional SRAM usage. This approach is common practice for deploying such cryptographic schemes on resource-constrained embedded platforms [BCH+23, CC21]. The primary drawback is an increased firmware size (approximately 390 KB), which leads to longer flashing times but does not affect the execution of the attack. The firmware is compiled using the build environment of the pqm4 library. The ChipWhisperer target operates at the standard clock frequency of 7.37 MHz.

4.2 Electromagnetic Injection Equipment

We adopt electromagnetic fault injection (EMFI) due to its more fine-grained controllability and higher repeatability compared to voltage or clock glitches. Precise tuning of timing, amplitude, pulse length, and probe position enables localized and reproducible disturbances of specific operations while minimizing unintended effects on the system. Other fault injection techniques, such as voltage glitching, have also been demonstrated to induce instruction-skipping faults on similar STM32 devices. However, these methods typically offer less spatial selectivity and fine-grained control than EMFI. Additionally, EMFI typically does not require physical modifications to the target device such as removing decoupling capacitors or tapping into power rails.

Electromagnetic pulses are generated using a ChipSHOUTER mounted on a motorized XYZ stage, which enables precise positioning of the injection probe above the target chip. Both the XYZ stage and the target device are fixed to an optical breadboard to ensure mechanical stability and repeatability. The ChipSHOUTER requires extremely short, low-voltage pulses to control the MOSFET that releases the high voltage charge through the injection tip. These pulses are usually on the order of tens of nanoseconds, requiring accurate control over both pulse width and timing to achieve reproducible fault injection. While the ChipWhisperer Lite can generate such pulses using its glitch module, its internal 96 MHz clock limits the timing resolution to 9.6 ns. Although using an external clock would improve this resolution, it would complicate subsequent use for power trace acquisition. Instead, we use a Raspberry Pi Pico (RP2040) executing a custom pio-assembly state machine at 200 MHz. The Pi Pico receives a trigger signal and then emits the specified length pulse at a precise offset. Both offset, and pulse length can be configured over serial with a resolution of 5 ns.

5 Attack Phases

First, we analyze the assembly implementation of the `memcpy` function call to determine its vulnerability to instruction-skipping faults. Next, we systematically explore the EMFI parameter space to identify configurations that reliably induce the targeted instruction skip on our device. Subsequently, we validate the efficacy of these configurations on the `memcpy` operation within the actual UOV implementation. The evaluation starts in a white-box setting with minor modifications to facilitate verification of successful fault injection. Finally, we demonstrate the complete attack on the unmodified pqm4 reference implementation by alternating between fault-free and faulted signature generations to derive the oil candidates necessary for secret key recovery.

5.1 Target Code Analysis

Listing 2 depicts the Assembly generated by the compiler for the `memcpy` function call. The code follows the ARM Architecture Procedure Call Standard (AAPCS), which specifies that the first three function arguments are passed via registers `r0`, `r1`, and `r2`. The instruction sequence begins by computing the destination address of the memcpy operation. Register

`r7` serves as the frame pointer for the current stack frame. The instruction computes the address of the local buffer `w`, which resides at a fixed offset within the stack frame. This address is temporarily stored in register `r3`. Next, the constant length of 68 bytes (`_V_BYTE`) is loaded into `r2`. Subsequently, the source pointer (vinegar) is loaded from the stack frame into register `r0` and the previously computed destination address is copied to `r1`. Finally, the `bl` ("Branch with Link") instruction transfers control to the memcpy implementation while storing the return address in the link register (`lr`).

Since our objective is to skip the entire `memcpy` operation, we target the corresponding branch instruction using fault injection. Specifically, we aim to identify fault parameters that suppress the execution of this branch while preserving correct execution of the surrounding code. The four instructions preceding the `memcpy` exclusively prepare its function arguments, and code following the `memcpy` does not depend on their correctness. Consequently, transient faults affecting these instructions do not impact the overall program behavior, assuming that the `memcpy` invocation itself is skipped. As a result, our attack is not strictly limited to faults that skip exactly one instruction. Faults causing the omission of up to four consecutive instructions remain effective, as long as the `memcpy` invoking branch instruction is skipped. Nevertheless, since the single instruction skip model is well established, we focus our following analysis on a fault that skips only the branch instruction.

5.2 Parameter Discovery

Effective EMFI requires the careful identification of interacting parameter settings that enable reliable and reproducible fault injection within a practical search effort.

To constrain the parameter space, we fix several parameters from the outset. While this may prevent us from identifying the best configuration, identifying a single functional configuration is adequate for our purpose. Throughout our experiments, we exclusively employ the standard "4 mm CW" injection tip supplied with the ChipSHOUTER (4 mm diameter, clockwise winding, ferrite core). The coil is kept at a constant Z-height as close as possible to the chip (approx. 0.1 mm distance). The target clock speed stays unchanged at 7.37 MHz.

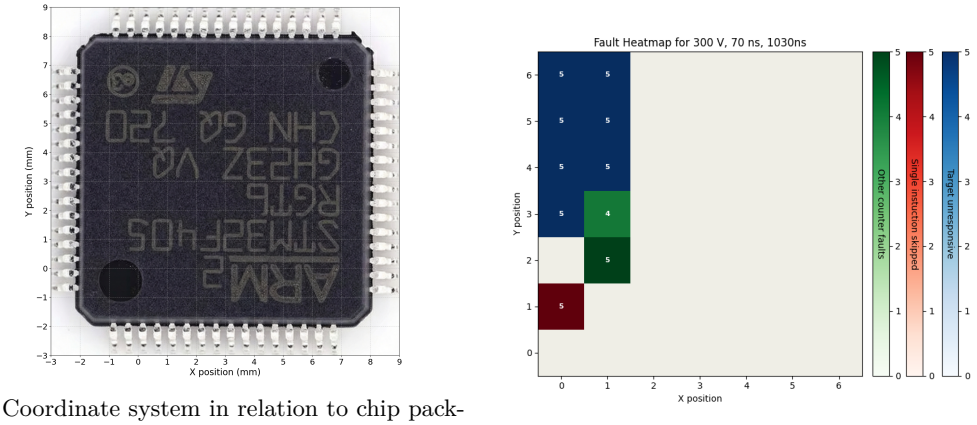
5.2.1 Glitch Characteristics

Since our objective is to induce a fault that effectively skips a single instruction, we first focus on profiling the glitch parameters that reliably produce this behavior. For this purpose, we implement a simple unrolled assembly loop that increments a register from 0 to 100, and seek EMFI configurations that suppress exactly one of the increment instructions.

The target device indicates the temporal boundaries of the test loop through a GPIO signal, which simultaneously serves as the trigger input for the Pi Pico-based pulse generator. The temporal offset of the injection pulse is adjusted to position it within the loop execution window. As the identity of the skipped instruction is not relevant at this stage of profiling, analysis of glitch parameter efficacy can be conducted without the need for temporal alignment to a specific target instruction. Nevertheless, we must account for

Listing 2: Assembly sequence for the `memcpy` invocation

```
0     add.w   r3, r7, #0x350      @ compute destination addr (&w)
      movs    r2, #68             @ load byte length (_V_BYTE)
2     mov     r1, r3              @ move destination addr
      ldr.w   r0, [r7, #0xb34]    @ load source pointer (vinegar)
4     bl      memcpy              @ branch with link to memcpy
```



(a) Coordinate system in relation to chip package

(b) Impact of glitch by position

Figure 1: Glitch results for: Voltage = 300 V, Pulse offset = 70 ns, Offset = 1030 ns

the well-established phenomenon that fault injection efficacy exhibits strong dependence on the intra-cycle timing of the electromagnetic pulse relative to the target clock signal. To address this temporal sensitivity, we perform a sweep of the glitch offset across one complete period of the target clock. Given the 7.37 MHz clock frequency, corresponding to a period of approximately 135.6 ns, we vary the offset within a 140 ns range at 10 ns intervals.

Spatial characterization is conducted by scanning the chip surface with an electromagnetic probe positioned at 49 locations arranged in a 7×7 grid with 1 mm spacing. For the electromagnetic pulse we concentrate exclusively on a limited number of parameters in order to expedite this profiling process. Three voltage levels (400 V, 350 V, 300 V) are tested in combination with 5 discrete pulse lengths (45, 50, 55, 60, 70 ns).

At each spatial position, all 15 glitch configurations are used in combination with the 14 glitch offsets for five fault injections respectively. To characterize the device behavior for each injected fault under various parameter sets, we employ a four-category taxonomy. The device returns the loop counter to the PC, where we use it to classify the outcome of each injection attempt according to the following definitions: A counter value of 100 indicates that no fault was injected and normal execution persisted. A value of 99 signifies that exactly one addition was skipped, which corresponds to the desired outcome. Any other counter value indicates that multiple instructions were skipped or that the control flow was otherwise corrupted. In instances where the target fails to return any value, we categorize it as unresponsive, as the fault likely induced a crash or other unexpected behavior.

We identify exactly one combination of position, voltage, pulse width, and offset that result in a consistent single instruction skip across all five iterations. Figure 1 illustrates the distribution of injected fault types per position using a voltage of 300 V, a pulse width of 70 ns, and an offset of 1030 ns. The data reveals that the area susceptible to single instruction skipping faults is extremely constrained, requiring precise positioning of the coil over the chip. While alternative parameter combinations enable fault injection at various positions, the configuration presented in Figure 1 was unique in achieving a 100% success rate across all five trial executions. Since five fault injections is a very small sample size, and as explained in Section 5.1 we are not strictly limited to single instruction skipping faults, we do not claim that this is the ideal parameter combination for our attack. Other suitable configurations may exist that offer comparable or even superior success rates.

5.2.2 Glitch Timing

The initial profiling stage yields a successful glitch configuration for skipping one of many addition operations, yet our attack requires precise targeting of a single branch instruction to bypass the full `memcpy` operation. Although the physical probe position and glitch characteristics remain transferable, the specific glitch timing requires recalibration to account for the different instruction type.

A new profiling program performs a 68 bytes `memcpy` from a source buffer to a target buffer, both initialized with known values. As previously, the EM glitch is triggered using a GPIO from the target. To enable triggering before the first instruction of the `memcpy` operation, 100 NOPs are performed in between the trigger signal and the `memcpy`, resulting in a fixed delay of approximately 13.5 µs. Using the before determined position and glitch characteristics, faults are injected with offsets from 14.9 µs to 22.5 µs at increments of 10 ns. Since branch and addition instructions differ significantly at the microarchitectural level, such as in the number of required clock cycles, we anticipate that the intra-cycle timing of the electromagnetic pulse must also be adjusted. Consequently, varying the glitch offset in increments of a complete clock period is insufficient.

Depending on the offset, the `memcpy` operation is affected in different ways by a glitch. It can be not affected at all, fully skipped, skipped partly or the target buffer experiences data corruption where bytes occur that do not originate from the initial values of the source or target buffer. This behavior can be explained by examining the assembly-level implementation of the `memcpy` routine (see Listing 2). When the glitch is injected too early (before the branch instruction) it disrupts the setup of the registers used as `memcpy` parameters. Skipping one of the parameter-initialization instructions causes the routine to reuse stale register contents as the source address, destination address, or copy length. Consequently, the `memcpy` may operate on unintended memory regions. When the glitch is injected too late, execution is already inside the `memcpy` function, so the fault occurs within the copy routine itself. In this scenario, the glitch impacts the internal control flow and data movement of the `memcpy` implementation rather than its setup. Such a fault can cause individual load or store instructions to be skipped or disrupt the loop-control logic. As a result, the copy loop may terminate prematurely, producing an incomplete `memcpy` where only part of the source buffer is transferred. In other cases, entire loop iterations may be skipped, creating gaps in the copied data. Skipping the `memcpy` operation can be achieved through multiple failure paths. For the purpose of our analysis, we do not distinguish between the specific internal causes and instead focus solely on the observable outcome. Specifically, we categorize the results into four classes:

1. **Unaffected:** The `memcpy` operation completes successfully, and the target buffer perfectly matches the source.
2. **Fully skipped:** The `memcpy` is entirely bypassed. The target buffer remains in its initial state with no data transferred.
3. **Partly skipped or Corrupted:** The operation results in a target buffer that matches neither the source nor the target buffer's pre-injection state.
4. **Crashed:** The device does not transmit the target buffer back to the host and becomes unresponsive, necessitating a hardware reset to restore functionality.

The results show, that between 15.5 µs and 16.1 µs multiple valid offsets exist that lead to full skipping of the memcpy operation with a high probability. Figure 2 shows the distribution of fault types for different glitch offsets, focusing on regions where the target buffer is affected. Offset ranges where the target buffer is not impacted have been omitted for clarity. For offsets greater than 16.37 µs, faults affecting the target buffer could still occur, but no faults were observed that skipped the memcpy as intended.

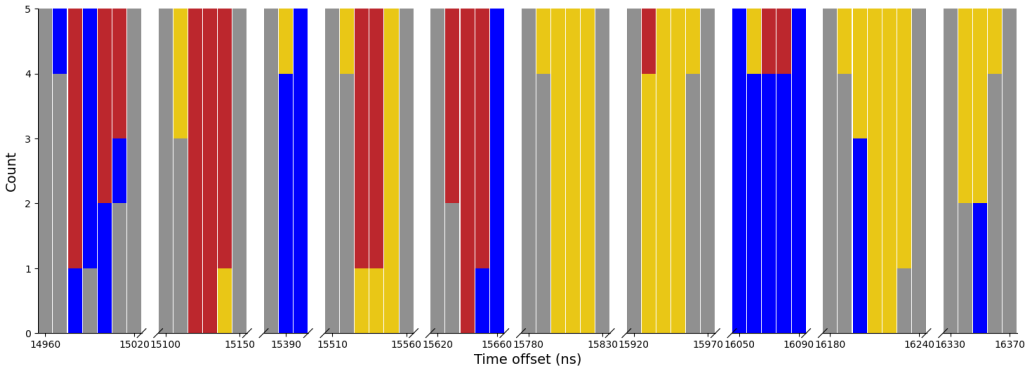


Figure 2: Standalone memcpy profiling: Target reaction to fault injection by glitch offset Gray = Unaffected ; Blue = Crashed ; Yellow = Partly skipped or Corrupted ; Red = Fully skipped

5.3 Whitebox Validation

To validate that the identified parameters remain effective within the context of a full UOV signature generation, we apply the established trigger configuration to the `memcpy` operation within the `ov_sign` function. We zeroize the signature buffer before every signature generation, ensuring that a successful fault results in a signature that directly exposes the oil vector. The generated signature is transmitted to the host, where the expected oil vector is derived using the reference `uov-Ip` implementation, by recalculating the signature using the same key, salt and message. A matching oil vector in the output signature indicates a successful fault, allowing efficient validation without performing full key recovery.

Our experimental results indicate that both the success rates and the effective timing window observed during the full `ov_sign` execution closely align with those obtained from the standalone `memcpy` profiling. Although the successful offsets are not perfectly identical—likely due to minor variations in the microarchitectural state, they consistently fall within the same narrow temporal range.

In a real-world scenario, an attacker would lack a dedicated trigger placed conveniently before the `memcpy` operation within the `ov_sign` routine. However, the execution time of the UOV signature generation is deterministic as long as no rejection sampling occurs. Using a trigger placed before the `ov_sign` call, we identify a narrow window of effective offsets (358 984 355–358 985 325 ns). At an offset of 358 984 420 ns, the attack achieves a success rate exceeding 90% over 500 fault injections. The distribution of observed fault outcomes is summarized in Table 1.

Table 1: Distribution of fault types for 500 attacks (300 V, 70 ns pulse length, 358 984 420 ns offset)

Fault Type	**Count**	**percentage**
Unfaulted	0	0%
Target unresponsive	1	0.2%
Otherwise invalid signature	40	8.0%
Oil detected in signature	**459**	**91.8%**
Total	500	100%

5.4 Practical Attack Execution

For the final attack, we reuse the previously identified glitch parameters and timing configuration. Triggering is implemented by setting a GPIO prior the `ov_sign` function call in the `uov-Ip` implementation. We remove the auxiliary zeroization of the signature buffer that was introduced during profiling. As a consequence, a successful fault does not directly yield the oil vector. Instead, it contains the bitwise exclusive-or of the oil value and the previous contents of the corresponding memory region. Since the same buffer is reused across consecutive signature generations, a simple bitwise XOR operation between the faulted signature and the previous signature reveals the oil vector as discussed in Section 3.

Therefore, our attack incorporates a two-step process: First we instruct the target to generate a valid signature without injecting a fault in order to initialize the signature buffer to a known state. Subsequently, the target generates a second signature during which the fault injection is performed.

In our threat model, the adversary has access to the public key and can therefore verify each generated signature to detect a fault. Upon observing an invalid signature, the corresponding oil candidate is derived via a bitwise XOR operation, after which the Kipnis–Shamir reconciliation attack is applied.

Applying the parameters and offset identified during the whitebox validation (see Table 1), we conducted the full attack over 300 signature generations and obtained 139 oil candidates. Among these, 126 were verified as correct and suitable for key recovery. Given that 146 of the 300 generations were intentionally fault-free signatures, this results in an effective glitch success rate of 81.8%.

6 Portability of the Attack

This section assesses the feasibility of mounting the proposed fault attack under realistic threat models. In particular, we discuss the challenges arising from limited attacker control over the target device and analyze how these constraints impact parameter discovery and attack transferability.

In a real-world setting, an attacker typically cannot deploy custom profiling code on the target device. Although access to the target firmware is a realistic assumption in many threat models through open-source releases, reverse engineering, or data leaks, flashing modified firmware is usually infeasible. Such an attempt violates secure boot protections or, in the case of the STM32F4, triggers memory erasure that destroys any resident secret key. Brute-forcing glitch parameters and timing is inefficient since validating attack success requires executing the computationally expensive reconciliation step.

Consequently, the feasibility of the attack critically depends on whether glitch parameters can be transferred from a separate training device, or whether the attacker can determine the required glitch parameters on the target device using more efficient techniques. We next consider the practical challenges associated with determining suitable fault injection parameters under these limitations.

6.1 Glitch Timing

Among the required fault injection parameters, the glitch offset presents the largest search space. A successful attack relies on the attacker's ability to trigger on an event originating from the target device that maintains a constant temporal offset to the vulnerable `memcpy` operation. Neither the content nor the length of the message impact the execution time of the `ov_sign` function, as the algorithm operates on a fixed-length hash digest of the message. Therefore, arbitrary signals like the device reset line, GPIO toggles, or UART communication activity can be used as triggers, as long as the execution time from the

trigger signal to the `ov_sign` function is predictable. It should be noted, however, that the length of the message affects the duration of the hash computation, which typically precedes the `ov_sign` function. Therefore, it is advantageous—though not strictly required—for the attacker to ensure that the message length is constant or known in advance.

If the attacker possesses the target firmware, identifying a suitable trigger and narrowing down the corresponding glitch offset to a single clock cycle is straightforward. The offset can be determined analytically by counting the instructions executed between the trigger event and the `memcpy` call, or empirically by measuring the timing in emulation or on a controlled test device.

Without knowledge of the target firmware, identifying a suitable offset becomes significantly more challenging. However, side-channel analysis or logical reverse-engineering techniques may still allow an attacker to narrow down the search window sufficiently to carry out a successful attack within a practical time frame.

6.2 Glitch Parameters

On a real world target without the ability to use dedicated profiling firmware, an attacker has the following options to search for instruction skipping glitch parameter combinations:

- **Brute Force:** Directly targeting the vulnerable `memcpy` to identify glitch parameters is highly inefficient, as it requires jointly searching the physical parameter space and the glitch offset, with success verification relying on the expensive Kipnis–Shamir reconciliation.
- **On-Device Profiling:** The attacker conducts a profiling phase on the target device by focusing on a simple, easily verifiable instruction rather than the vulnerable `memcpy`. For example, glitching a GPIO toggle allows immediate detection of instruction skipping without invoking the computationally expensive Kipnis–Shamir reconciliation. While more efficient than brute force, it remains more challenging than the dedicated profiling firmware scenario described in Section 5.
- **Replica Profiling:** The attacker determines the optimal glitch parameters on a controlled training device using dedicated profiling firmware and attempts to transfer them to the target device.

6.3 Real-World Scenario

We consider a realistic attack scenario in which an STM32F4 microcontroller uses the pqm4 implementation for generating UOV signatures. Readout protection is enabled, preventing direct extraction of the secret key from flash memory, while the device produces signatures on arbitrary inputs, either periodically or on demand.

The memory region used for signatures is reused across successive executions without reinitialization. Consequently, observing the previous signature fully determines the memory state before the next signature generation. The adversary can employ a training device with identical hardware and leverage a trigger signal that maintains a fixed temporal offset to the `ov_sign` function call.

6.4 Attack Transfer via Replica Profiling

In this work, we follow a replica profiling approach. The methodologies detailed in Section 5.2 constitute the training phase on our replica device. Leveraging knowledge of the trigger signal timing and the open-source pqm4 implementation, we calculated the approximate glitch offset for the vulnerable `memcpy` operation. Section 5.3 described

fine-tuning this offset using known keypairs for direct oil candidate verification, avoiding expensive Kipnis-Shamir reconciliation during calibration.

With the ChipWhisperer target from Section 5 acting as our training device, the next phase involves replicating these results on a second device that represents our actual attack target. Ultimately, the feasibility depends on the portability of the attack, specifically whether the glitch parameters found on the training device remain effective on the actual target device. We replace the STM32F415 ChipWhisperer target on the CW308 UFO Board. Due to hardware availability, we utilize an STM32F405 target board as a substitute which is architecturally equivalent. Synchronization is achieved using the identical trigger mechanism employed during the profiling phase. While the message to be signed is hardcoded in our experimental setup, this is not a prerequisite for a successful attack.

Initially, we apply the exact glitch configuration and offset derived from the replica device in Section 5.3. To account for minor temporal variances observed during preliminary testing, we define a scan window covering approximately ±50 ns around the reference offset. The specific injection parameters are configured as depicted in Table 2. For every offset, we attempt to fault 50 signature generations and utilize the public key to verify the results subsequently. During this initial test, all generated signatures remain valid. We observe neither successful fault injections nor any device crashes or resets, indicating that the target remains stable under these specific parameters. The total runtime required to generate the 500 signatures on the target and validate them on our x86 host system is 17 minutes. The complete absence of faults or instability suggests that the underlying issue is related to the glitch characteristics rather than its timing.

Our previous position profiling (see Figure 1) indicated that the vulnerable area on the chip is highly localized, spanning less than 1×1 mm. Upon swapping the target boards, we rely on the mechanical alignment of the CW308 UFO board to place the new chip in the identical position. However, manufacturing tolerances in the PCB assembly or slight variations in chip soldering could result in a misalignment of the chip. Consequently, we expand our attack phase to include a spatial search, varying the coil position by ±0.5 mm in X and Y direction around the original coordinates. At each spatial position, we inject 20 faults for every tested offset. Table 3 summarizes the aggregated results per position. The complete signature generation and validation phase requires 41 minutes.

Based on the identified coordinates and the 358 984 430 ns offset, we execute our primary attack script which orchestrates the generation of both correct and faulted signatures in

Table 2: Experimental Parameters

Voltage [V]	**Pulse length** [ns]	**Coil position** (x, y)	**Time offset** [ns]
300	70	$(0, 1)$	range $(358984400, 358984500, 10)$

Table 3: Signature validation results by position for 200 attempts

Coil Position (x, y)	**Valid Sigs**	**Invalid Sigs**	**Crashes**
(-0.5, 0.5)	200	0	0
(-0.5, 1)	200	0	0
(-0.5, 1.5)	186	**13**	1
(0, 0.5)	200	0	0
(0, 1)	200	0	0
(0, 1.5)	195	3	2
(0.5, 0.5)	15	0	185
(0.5, 1)	200	0	0
(0.5, 1.5)	198	0	2

an alternating sequence. Performing a bitwise XOR between a faulted signature and its preceding correct signature allows for the extraction of an oil candidate. The generation of 10 oil candidates (10 faulted signatures) completes in under five minutes and requires the calculation of only 36 signatures. We subsequently apply the Kipnis-Shamir reconciliation attack to these candidates to recover the secret key. Overall, 5 out of the 10 generated oil candidates prove valid. Since processing a single candidate takes approximately 34 minutes, the two required attempts contribute an additional 68 minutes to the overall attack time.

In total, we successfully transitioned from profiling on our reference device to a complete key recovery on the target device in less than three hours. Only minor adjustments to the glitch parameters were required, which could also be attributed to mechanical tolerances of our test setup. We did not conduct a large-scale statistical analysis to determine the precise success rate on the target device. However, the 5 successful key recoveries in such a short timeframe demonstrate that replica profiling is a viable approach and can drastically reduce the parameter search space.

7 Countermeasures

In this section, we discuss potential countermeasures against skip-addition fault attacks targeting the signature generation procedure. The vulnerability demonstrated in the previous sections relies on two fundamental weaknesses: the absence of fault-detection mechanisms and the predictability of the memory region used for the oil-vinegar combination. Consequently, we identify two main classes of countermeasures, which are discussed below. The first class ensures that invalid signatures are never returned, while the second class prevents memory predictability and therefore extraction of the oil vector.

7.1 Signature Validation

The initial and most straightforward method is to perform a verification of each generated signature to confirm its authenticity. If the produced signature σ matches the intended output ($\sigma = \sigma_g$), it is returned; otherwise ($\sigma = \sigma_f$), independent random values are output. This approach has been previously discussed for deterministic UOV variants [BSK25]. While effective, it incurs additional computational overhead due to the extra verification step.

7.2 Variable Random Initialization

The proposed attack relies on the fact that the content of the signature buffer is predictable before the oil is added onto it. This predictability allows an attacker to reverse the `gf256v_add` operation and extract the oil. In many implementations, including the one studied here, this predictability is a direct result of the signature pointer being reused for multiple signature generations without intermediate clearing.

A robust countermeasure is randomization of the signature buffer immediately before every signature generation. By filling the signature memory area with cryptographically secure random data, an attacker can no longer extract the oil vector through a bitwise XOR with the previous signature.

While one could argue that this responsibility lies with the end-user of the library, we believe such an approach is insufficient. It is common for users of standardized cryptographic libraries to integrate these tools without possessing a deep understanding of the underlying implementation or the specific threats posed by fault injection. Implementing this countermeasure directly within the library ensures that the system remains secure regardless of the user's familiarity with internal mechanics.

7.3 Reorganization of Code

If the computational overhead of 7.1 and 7.2 is not acceptable, we also propose two countermeasures that only require slight reorganizations of the existing code and no additional memory or computational resources.

The success of our attack relies on the in-place mixing of oil and vinegar variables within the persistent signature buffer, whose reuse across generations renders its content predictable. To mitigate this, we propose decoupling the signature buffer from the mixing process to ensure it never holds isolated oil or vinegar components. By using only local variables as operands for `gf256v_add`, any skipped assignment forces the operation to utilize stale data from the program stack. Unlike the persistent signature buffer, data on the stack is inherently volatile and significantly harder to predict. This uncertainty prevents the establishment of the known baseline required to extract the oil vector. We outline two specific strategies to implement this decoupling below.

Modification of the `gf256v_add` Function The first strategy mitigates the risk by modifying the `gf256v_add` function to support a three-operand interface. In the original design, the first parameter acts simultaneously as the input accumulator and the result destination. As shown in Listing 3, we introduce a separate output parameter to store the result. Therefore, the XOR is performed on the local `y` and `vinegar` variables and only the finished result is written to the signature.

Performing Mixing in the Vinegar Variable A second strategy achieves the same decoupling without requiring changes to the `gf256v_add` function prototype. Instead of modifying the arithmetic routine, we reorder the high-level logic as illustrated in Listing 4. The XOR addition is performed in-place on the local `vinegar` variable rather than the external signature buffer, cf. Listings 4 line 7. The fully mixed result is only copied to the signature pointer once the operation is complete, also effectively isolating the sensitive mixing process within the local stack frame.

7.4 Discussions on Set/Reset-based Faults

So far, we've mostly looked at fault injections through EMFI, which adds instruction skips while the code is running, since we focused on the skip addition attack reported in [ACK25]. However, it is also feasible to generate set or reset faults on an ARM platform with an EMFI setup, as detailed in [MDH+13, RBSG23]. Subsequently, we would like to discuss additional fault injection countermeasures and also prevent this type of attack.

When considering this type of fault attack, an attacker could set the vinegar variable to zero during the generation of the vinegar variables to establish a deterministic signature

Listing 3: Countermeasure using modified `gf256v_add` function with dedicated output variable

```
0  int ov_sign( uint8_t *signature, const sk_t *sk, const uint8_t *message, size_t mlen )
       {
       // ...
2      uint8_t vinegar[_V_BYTE];
       uint8_t y[_PUB_N_BYTE];
4      // ...
       hash_final_digest( vinegar, _V_BYTE, &h_vinegar); // Calculate vinegar
6      // ...
       uint8_t *w = signature;
8      gfmat_prod(y, sk->O, _V_BYTE, _O, x_o1 ); // Calculate oil
       gf256v_add(w, vinegar, y, _V_BYTE ); // Mix oil and vinegar
10     // ...
   }
```

generation setting. For instance, an attacker could accomplish this by introducing a reset fault injection during the assignment of the easing result to the vinegar variable as the final procedure after the final hash calculation for the vinegar generation, which is facilitated by a hash function call. See Listings 1 in line 5. In this attack scenario, the oil values would only be added to the signature with our proposed countermeasure, as demonstrated in Section 7.3. In this scenario, an attacker could directly extract the oil variables from the signature by injecting a single fault into a signature generation with an arbitrary message. Another option to inject a set- or reset-fault would be during the computation of the 0 part of T, cf. Listing 4 in line 6.

This attack would also be effective when the injection of a set fault type is taken into account, in which all values of the targeted variable are set to one. In this instance, the faulted signature contains the bitwise inverse oil values. Therefore, before initiating the reconcilability attack, the attacker must first invert the value. However, attacking the addition of mixing oil and vinegar cannot be exploited because it would directly put only zeros into the signature.

To prevent this exploitation by a set- or reset-fault injection attack, it is necessary to verify that the vinegar values are not zero, or that all bits are not set to one before the mixing of the vinegar values with the oil values.

The verification of whether the vinegar variables are set to zero or one can be verified if the vinegar variables are byte-wise XOR with the next adjacent byte from the first to the last one. If after the last XOR operation the result is zero, then all bits in the vinegar variable are identical, meaning either zero or one, and this would indicate that set- or reset-fault injection has been conducted before. Listings 5 depicts an exemplary implementation to check if a set or reset-fault has been injected. This check function can be place just before the addition operation for mixing the oil and vinegar variables, for instance before line 7 in Listings 4.

The exploitation of the side-channel leakage of the XOR operation of the two adjacent bytes is challenging since the two bytes should be completely random if they are genuine vinegar variables. Additionally, the vinegar variables are rendered invalid for the subsequent run in a non-deterministic signature generation environment if they are utilized for a single run. Therefore, it is exceedingly unlikely that the operations of this check function will emit exploitable side-channel analysis in order to extract the oil values, to the best of our knowledge.

Please be advised that the addition function `gf256v_add` should not be executed with the oil values in the event of zero value detection. This is due to the possibility of exploitable side-channel leaks. This supplementary information could be utilized to ascertain the oil values and reconstruct them using a methodology similar to that employed in [ACK+23].

Listing 4: Reorganization of the code to only assign the sum of **v** and **o** to the pointer `w` refering to the memory location of the siganture

```
0  int ov_sign( uint8_t *signature, const sk_t *sk, const uint8_t *message, size_t mlen )
       {
       // ...
2      uint8_t *w = signature;    // [_PUB_N_BYTE];
       // identity part of T.
4      memcpy( w + _V_BYTE, x_o1, _O_BYTE );
       // Computing the O part of T.
6      gfmat_prod(y, sk->O, _V_BYTE, _O, x_o1 );
       gf256v_add(vinegar, y, _V_BYTE );
8      memcpy( w, vinegar, _V_BYTE );
       //...
10 }
```

7.5 Additional Countermeasure Considerations

To increase the effort for an attacker in the perturbation phase of a fault injection attack, additional measures can be implemented next to the fault detection mechanism. In addition to the heuristic time jitter that is introduced during the signature generation phase as a result of the rejection sampling, a constant random jitter [DFM+11] can be implemented to further increase the effort required during the fault injection proving phase to identify appropriate pitch parameters. Another way to increase the effort required to extract sufficient oil values by a skip-addition attack is to prevent all vinegar values from being added to the oil vectors at once by a separate function. Using the MAYO reference implementation[5] as a guideline, the system can be solved, and the oil and vinegar values can be added in a chunk-by-chunk way using a loop structure. Thus, it is challenging to extract multiple vinegar or oil values with a single skip addition attack, necessitating multiple executions. It is also possible to previous mentioned countermeasures into this interleaved method for generating signatures, as demonstrated in [ACK25].

8 Conclusion

In this paper, we present a practical electromagnetic fault injection attack against UOV-based signature schemes that enables complete secret key recovery by faulting a single signature. To execute this attack in a real-world setting, the attacker requires knowledge of the signature buffer's memory content prior to the signature generation. This prerequisite is typically satisfied in scenarios where the memory is initialized to a known default value or where the same memory region is reused for multiple signature generations, allowing the previously generated signature to serve as the known initial state. Skipping a single memcpy operation causes the oil variables to leak into the signature. This compromised output can then be processed with the reconciliation attack to extract the secret key in about 30 minutes. Contrary to existing fault attacks targeting deterministic variants or requiring multiple injections, our skip-addition attack, based on exploiting memory re-use vulnerabilities, succeeds against the current non-deterministic pqm4 implementation.

We validate the attack on ARM Cortex-M4 targets using EM fault injection to induce instruction skipping faults and detail the strategic profiling techniques employed to efficiently identify effective glitch parameters. Without claiming to have identified the best glitch parameters, we managed to reach a success rate of over 90% in a short timeframe.

Critically, we demonstrate the practical feasibility of the attack under realistic real-world conditions. By porting parameters from an STM32F415 training device to an STM32F405 attack target with minimal spatial adjustments, we complete the entire transfer process to successful key recovery in less than three hours. The ability to reuse glitch parameters across devices streamlines the profiling phase in real-world scenarios and greatly increases the practical threat level of the attack.

[5] https://github.com/PQCMayo/MAYO-C/blob/main/src/mayo.c

Listing 5: Exemplary implementaiton of checking if all bits in a bit sting are identitical by a byte wise check)

```
int check(uint8_t){uint8_t a*, unsigned_num_byte}
{
        while (_num_byte -1)
        {
                a[_num_byte]^=[_num_byte + 1];
        }
return  a;
}
```

We identify several viable countermeasures including signature validation, code reorganization eliminating memory reuse, and initialization/masking with fault detection. Our analysis shows that masking combined with fault detection provides the strongest defense while requiring careful implementation to avoid side-channel vulnerabilities.

This work demonstrates that physical security remains critical for multivariate signature schemes advancing through NIST standardization, particularly on resource-constrained platforms where memory optimization may create exploitable vulnerabilities.

References

[ACK+23] Thomas Aulbach, Fabio Campos, Juliane Krämer, Simona Samardjiska, and Marc Stöttinger. Separating oil and vinegar with a single trace side-channel assisted Kipnis-Shamir attack on UOV. IACR Transaction of Cryptographic Hardware and Embedded Systems, 2023(3):221–245, 2023.

[ACK25] Thomas Aulbach, Fabio Campos, and Juliane Krämer. Sok: On the physical security of uov-based signature schemes. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, Post-Quantum Cryptography, pages 199–231, Cham, 2025. Springer Nature Switzerland.

[AKKM22] Thomas Aulbach, Tobias Kovats, Juliane Krämer, and Soundes Marzougui. Recovering rainbow's secret key with a first-order fault attack. In Lejla Batina and Joan Daemen, editors, AFRICACRYPT, pages 348–368, Cham, 2022. Springer Nature Switzerland.

[AMSU24] Thomas Aulbach, Soundes Marzougui, Jean-Pierre Seifert, and Vincent Quentin Ulitzsch. Mayo or may-not: Exploring implementation security of the post-quantum signature scheme mayo against physical attacks. In Workshop on Fault Detection and Tolerance in Cryptography, pages 28–33, 2024.

[BCC+23] Ward Beullens, Fabio Campos, Sophía Celi, Basil Hess, and Matthias J. Kannwischer. MAYO. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[BCD+23] Ward Beullens, Ming-Shing Chen, Jintai Ding, Boru Gong, Matthias J. Kannwischer, Jacques Patarin, Bo-Yuan Peng, Dieter Schmidt, Cheng-Jhih Shih, Chengdong Tao, and Bo-Yin Yang. UOV. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[BCH+23] Ward Beullens, Ming-Shing Chen, Shih-Hao Hung, Matthias J. Kannwischer, Bo-Yuan Peng, Cheng-Jhih Shih, and Bo-Yin Yang. Oil and vinegar: Modern parameters and implementations. IACR Transaction on Cryptographic Hardware and Embedded Systems, 2023(3):321–365, 2023.

[Beu21] Ward Beullens. Improved cryptanalysis of UOV and rainbow. In Anne Canteaut and François-Xavier Standaert, editors, Advances in Cryptology - EUROCRYPT, volume 12696 of Lecture Notes in Computer Science, pages 348–373. Springer, 2021.

[Beu22] Ward Beullens. Breaking rainbow takes a weekend on a laptop. In Yevgeniy Dodis and Thomas Shrimpton, editors, Advances in Cryptology - CRYPTO, volume 13508 of Lecture Notes in Computer Science, pages 464–479. Springer, 2022.

[BSK25] Sven Bauer, Fabrizio De Santis, and Kristjane Koleci. Fault attacks against UOV-based signatures. IACR Cryptol. ePrint Arch., 2025.

[CC21] Ming-Shing Chen and Tung Chou. Classic mceliece on the ARM cortex-m4. IACR Transaction on Cryptographic Hardware and Embedded Systems, 2021(3):125–148, 2021.

[DFM+11] Jean-Max Dutertre, Jacques J.A. Fournier, Amir-Pasha Mirbaha, David Naccache, Jean-Baptiste Rigaud, Bruno Robisson, and Assia Tria. Review of fault injection mechanisms and consequences on countermeasures design. In Design & Technology of Integrated Systems in Nanoscale Era, pages 1–6, 2011.

[DGG+23] Jintai Ding, Boru Gong, Hao Guo, Xiaoou He, Yi Jin, Yuansheng Pan, Dieter Schmidt, Chengdong Tao, Danli Xie, and Ziyu Yang, Bo-Yin Zhao. TUOV. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[FIH+23] Hiroki Furue, Yasuhiko Ikematsu, Fumitaka Hoshino, Tsuyoshi Takagi, Kan Yasuda, Toshiyuki Miyazawa, Tsunekazu Saito, and Akira Nagai. QR-UOV. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[FKNT22] Hiroki Furue, Yutaro Kiyomura, Tatsuya Nagasawa, and Tsuyoshi Takagi. A new fault attack on uov multivariate signature scheme. In Jung Hee Cheon and Thomas Johansson, editors, Post-Quantum Cryptography, pages 124–143, Cham, 2022. Springer International Publishing.

[GCF+23] Louis Goubin, Benoît Cogliati, Jean-Charles Faugére, Pierre-Alain Fouque, Robin Larrieu, Gilles Macario-Rat, Brice Minaud, and Jacques Patarin. PROV. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[HTS11] Yasufumi Hashimoto, Tsuyoshi Takagi, and Kouichi Sakurai. General fault attacks on multivariate public key cryptosystems. In Bo-Yin Yang, editor, Post-Quantum Cryptography, pages 1–18, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[JD24] Sönke Jendral and Elena Dubrova. MAYO key recovery by fixing vinegar seeds. IACR Cryptol. ePrint Arch., page 1550, 2024.

[KL19] Juliane Krämer and Mirjam Loiero. Fault attacks on uov and rainbow. In Ilia Polian and Marc Stöttinger, editors, Constructive Side-Channel Analysis and Secure Design, pages 193–214, Cham, 2019. Springer International Publishing.

[KRSS19] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4. IACR Cryptol. ePrint Arch., page 844, 2019.

[MDH+13] Nicolas Moro, Amine Dehbaoui, Karine Heydemann, Bruno Robisson, and Emmanuelle Encrenaz. Electromagnetic fault injection: Towards a fault model on a 32-bit microcontroller. In Workshop on Fault Diagnosis and Tolerance in Cryptography, pages 77–88, 2013.

[MIS20] Koksal Mus, Saad Islam, and Berk Sunar. QuantumHammer: A practical hybrid attack on the LUOV signature scheme. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, ACM CCS 2020, pages 1071–1084. ACM Press, November 2020.

[OBG+23] Santiago Arranz Olmos, Gilles Barthe, Ruben Gonzalez, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, and Peter Schwabe. High-assurance zeroization. IACR Cryptol. ePrint Arch., 2023.

[Pat97] Jacques Patarin. The oil and vinegar signature scheme, 1997.

[PCF+23] Jacques Patarin, Benoît Cogliati, Jean-Charles Faugére, Pierre-Alain Fouque, Louis Goubin, Robin Larrieu, Gilles Macario-Rat, and Brice Minaud. VOX. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[Pqo25] Pqov/pqov. github, December 2025.

[RBSG23] Jan Richter-Brockmann, Pascal Sasdrich, and Tim Güneysu. Revisiting fault adversary models – hardware faults in theory and practice. IEEE Transactions on Computers, 72(2):572–585, 2023.

[SK20] Kyung-Ah Shim and Namhun Koo. Algebraic fault analysis of uov and rainbow with the leakage of random vinegar values. IEEE Transactions on Information Forensics and Security, 15:2429–2439, 2020.

[SMA+24] Oussama Sayari, Soundes Marzougui, Thomas Aulbach, Juliane Krämer, and Jean-Pierre Seifert. Hamayo: A fault-tolerant reconfigurable hardware implementation of the MAYO signature scheme. In Constructive Side-Channel Analysis and Secure Design, volume 14595 of Lecture Notes in Computer Science, pages 240–259. Springer, 2024.

[WCD+23] Lih-Chung Wang, Chun-Yen Chou, Jintai Ding, Yen-Liang Kuan, Ming-Siou Li, Bo-Shu Tseng, Po-En Tseng, and Chia-Chun Wang. SNOVA. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.